Measuring Data Exposure in LLM-Integrated Productivity Tools

Marawan Eldeib

University of Stuttgart
Supervisor: Prof. Dr. Marco Aiello
September 22, 2026

***Abstract*—Large-language-model (LLM)-integrated writing assistants run as browser extensions that observe the text a user types or pastes into web-page fields and send it to their servers for analysis. We measure how much of a confidential document such tools transmit by default, in a scenario where the user has not chosen to share it with any AI tool, using an HTTPS man-in-the-middle proxy and a synthetic memo carrying twelve unique planted identifiers, including a UUID canary. In the primary experiment we compare two browser extensions under identical paste conditions: across five runs each, Grammarly and LanguageTool transmitted essentially the entire document (99.0% and 91.9% of characters) and all twelve secrets, including the canary, while a no-extension baseline transmitted none. A secondary host-phase measurement, under different interaction conditions, shows the exposure is surface-dependent: opening the file in Microsoft Word with connected experiences enabled, with no paste and no submission, streamed 96.8% of it to a Microsoft cloud endpoint, and disabling a single Office setting removed that channel in a single-run check. Across the conditions we tested, content leaves the machine when it enters an LLM-integrated surface, independent of where the file is stored; we frame this as a gap between default behaviour and reasonable user expectation rather than an exploit. All test data is synthetic and the methodology is reproducible.**

***Index Terms*—Browser extensions, data exposure, HTTPS interception, LLM-integrated productivity tools, PII leakage**

I. Introduction

Large-language-model (LLM) integrated productivity tools (grammar checkers, paraphrasers, and "AI assistant" extensions) have become a default part of many users' browsers. They work by observing the content of text fields and sending that content to remote servers for analysis, then returning suggestions. The convenience is real; the privacy cost is largely invisible. A user who has been handed a confidential file and told "do not share this with any AI tool" would not knowingly paste it into a chatbot. But that same user may have a grammar-checking extension enabled in the background, reading every field they type or paste into. We ask the following measurement question:

> How does user data exposure differ across LLM-integrated productivity tools during controlled use?

When a confidential document reaches such a tool—whether the user invokes it on demand, or an enabled extension captures a field they typed into for an unrelated task—how much of the document leaves the machine, and does it include the specifically sensitive parts? We distinguish these two cases, active use and incidental exposure, in Section II. Grammar-checking browser extensions are the entry point, but the study spans the integration types these tools take in practice: browser extensions, desktop and editor applications, and system-level services. The controlled, same-task comparison is limited to the browser extensions; the host phase is a secondary, exploratory extension under different interaction conditions. We use "LLM-integrated productivity tools" for products that provide AI/LLM-assisted functionality; our network measurements establish remote transmission of document content, not whether a particular captured request was processed by an LLM internally.

This is not a search for a vulnerability. The tools examined here are doing exactly what they were designed to do. We find that, under identical conditions, both browser tools transmit almost the entire document and every planted secret while a no-extension baseline transmits nothing, and that across desktop and system surfaces the same exposure tracks whether content enters an LLM-integrated surface rather than where the file is stored. Our contribution is a reproducible measurement of this default behaviour and an analysis of how it diverges from users' expectations. Our contributions are:

- A reproducible HTTPS-interception method with a unique UUID canary, applied to Grammarly and LanguageTool against a no-extension baseline.
- A host-level extension showing exposure is surface-dependent: it depends on the surface a document enters, not on where the file is stored, with negative controls (USB, iCloud, non-AI viewers) and a one-setting mitigation for the Word transmission.
- A permission analysis showing the exposure channel is all-sites text access, not the microphone or camera.

II. Background and Threat Model

A. How these tools work

A writing-assistant extension registers content scripts that attach to a page's editable fields, that is, text boxes and rich-text areas (in web terms, `<textarea>` and `contenteditable` elements), on visited pages. When text appears in such a field, the extension transmits it, typically over HTTPS or a WebSocket, to its backend, which runs the analysis and returns suggestions. Because the analysis is

server-side, the text itself must leave the user's machine for the feature to function at all.

B. Intended use versus incidental exposure

A user's text can reach one of these tools in two ways, and this study measures both. In active, on-demand use, the user knowingly hands the text over: they type into the tool's own editor, or click an action such as "paraphrase," "summarise," or "translate." Transmission is expected, but its extent is not: how much of a document a tool ships once it is invoked is itself a privacy-relevant quantity, which we measure for a desktop translator and an on-demand assistant. In incidental exposure, the extension is installed and enabled, and it captures text the user entered for an unrelated purpose, with no intent to use the tool and no action beyond typing or pasting. Both cases matter for confidential work: the first because users routinely underestimate how much a tool sends even when they choose to use it, the second because the disclosure happens with no step the user would recognise as sharing. Having established the transmission mechanism above, we measured both interaction classes across a set of tools, reporting each condition by its class.

C. Threat model

We model an ordinary, non-adversarial situation:

> A user receives a confidential file and is explicitly told not to share it with AI tools. They have a writing-assistant extension installed and enabled by default. In the course of normal work they paste part of the document into a normal text field (an email reply, a web form, a notes box). Without any further action, the extension transmits the content to its servers.

We do not assume a malicious vendor, a compromised endpoint, or a network attacker; the concern is unintended disclosure of document content to remote service providers through enabled productivity-tool integrations. The paste here is for an unrelated task, an email reply and not a request to any AI, yet the enabled extension has already transmitted the content. This incidental paste is our primary scenario, but it is one point on a spectrum we measure: from active, on-demand use where the user does invoke a tool, through this incidental case during normal work, to purely passive exposure where opening a received file transmits it (Section V-E). The more confidential the work, the more each of these situations matters.

III. Related Work

The privacy risks of browser extensions have been studied for the better part of a decade. Starov and Nikiforakis [1] were among the first to systematically measure the "privacy diffusion" enabled by extensions, showing that many leak identifying information far beyond what their function requires. Weissbacher et al. [2] built Ex-Ray to detect history-leaking extensions, and Aggarwal et al. [3] analysed and detected "spying" extensions that exfiltrate (covertly transmit) browsing behaviour. Chen and Kapravelos [4] scaled dynamic taint tracking to 181,683 extensions, flagging thousands that leak privacy-sensitive information. These works established that extension data collection is widespread and frequently undisclosed. They also introduced the complementary detection methods the field still relies on: differential string search, content-agnostic network analysis, and taint tracking. Static data-flow analysis (e.g. DoubleX [5]) later added scale on the code side. This line of work rests on a longer study of the extension security model itself: Carlini et al. [6] evaluated Chrome's permission and isolation architecture and showed that over-broad host permissions undercut its guarantees, and Pantelaios et al. [7] showed that benign extensions can turn malicious through later updates, which is why a runtime, network-level measurement of what a tool actually transmits is more reliable than trusting a single reviewed snapshot.

More recent large-scale studies have moved from browsing metadata to page content. Nayak et al. [8] experimentally analysed sensitive-data access by extensions, including access to credentials on login pages, across the Chrome Web Store. Xie et al. [9] built Arcanum, a dynamic taint-tracking system, and found that thousands of extensions, used by an estimated 144 million people, collect sensitive on-page content such as email bodies, social-media activity, and banking information, often without accurate disclosure. Closer to our own design, Bui et al. [10] intercepted the network traffic of roughly 47,000 extensions and compared the observed outbound data flows against each extension's stated privacy practices, finding hundreds whose transmissions contradicted their disclosures. From the tracking side, Dambra et al. [11] measured web tracking from the user's perspective, building on the browser-instrumentation measurement paradigm established by OpenWPM [12].

The most behaviourally similar precedent is "read-before-submit" exfiltration: Senol et al. [13] showed that email addresses and even passwords are sent to third parties as the user types, before any form is submitted; Munir et al. [14] recently generalised this, finding widespread third-party keystroke event-listener interception across popular sites, and Starov et al. [15] earlier quantified personally identifiable information (PII) leaking from website contact forms. We extend that "transmitted before the user acts" framing from trackers to writing-assistant extensions. Methodologically, our detection step, searching intercepted flows for known sensitive strings, follows a mature network-level PII-measurement lineage rooted in dynamic taint tracking (TaintDroid [16]) and network-level flow analysis over decrypted mobile traffic (ReCon [17]). Because outbound payloads can be encoded or obfuscated, we adopt unique planted canaries in the spirit of the black-box differential leak detection introduced by AGRIGENTO [18], so that any transmitted fragment is unambiguously attributable to our input.

Closest to the present work, Vekaria et al. [19] audited nine generative-AI browser assistants and, using traffic decryption, showed that several forward full webpage content, including medical and banking details, to their own servers and to third-

party trackers. Our study is narrower and complementary: rather than a broad crawl, we perform a deep, controlled, and fully reproducible measurement of a specific, common threat model (a confidential document pasted into a field with a grammar checker running by default), using planted canaries that make each leak individually and unambiguously attributable.

Finally, why the transmitted content matters is reinforced by a parallel line of work on the privacy risks of large language models, which increasingly sit behind these assistants. Carlini et al. [20] showed that language models memorise and can regurgitate verbatim personal data from their training set, and Nasr et al. [21] extended such extraction to aligned, production models, recovering training examples from deployed systems such as ChatGPT; Lukas et al. [22] quantified PII leakage from language models and showed that scrubbing and differential privacy leave residual exposure. Staab et al. [23] showed that models can infer sensitive attributes (location, age, income) from ordinary prose; and Mireshghallah et al. [24] showed that LLM-integrated applications routinely disclose information to recipients that are inappropriate for the context. Text sent to a writing assistant is therefore not inert once it leaves the machine, which is precisely what makes quantifying how much of it leaves, and to whom, worthwhile. This gap between what users believe about their exposure and what actually leaves their devices has itself become a subject of study: recent work frames a "privacy paradox" between user perceptions and the reality of PII leakage in LLM-integrated settings [25], motivating the concrete, quantified measurement we provide here.

Once the text leaves the machine, it can be kept and reused by others. Threat-intelligence reporting has found deployed AI services and unauthorised "model-reseller" proxies silently storing and passing on users' inputs, including names, contact details, and corporate data, without their knowledge [26]. Industry analyses report the same for the tools we study: a Unit 42 investigation found "productivity" browser extensions that read users' email as they write it and capture page content [27], and enterprise-security reporting notes that such extensions read page content largely invisibly to standard data-loss-prevention tools, with much of the data coming from personal, unmonitored accounts [28, 29].

Research gap. Prior work either counts how many extensions leak at scale or audits AI assistants broadly; what is missing is a controlled, canary-proven, per-secret measurement of how much of a single confidential document leaves the machine, across both the browser extensions and the everyday host applications (Office, desktop clients, cloud-AI viewers) a real user actually encounters. This paper fills that gap.

IV. Methodology

A. Study design and conditions

The study has two phases. The *primary* experiment is a controlled, same-task browser comparison of three conditions: Grammarly (the always-on grammar and style checker that motivated this project), LanguageTool (a grammar checker from a different company, LanguageTooler GmbH, based in Germany, that needs no account, a clean second tool for checking whether the behaviour repeats), and a no-extension baseline (the same browser and steps with no extension installed, used as a negative control for the browser's background traffic). Testing two independent tools rather than one gives stronger evidence: if both behave the same way, the result is not a quirk of a single product. The *secondary*, exploratory host phase then probes several other surfaces: the DeepL desktop client, Microsoft Word with Office connected experiences, the Grammarly desktop client under Protected View, Edge/Copilot on a PDF, and passive storage or non-AI viewers; these conditions differ in user action and file format, so they are read individually rather than as a same-task cross-tool comparison. Together the conditions cover the browser-extension, desktop-application, and editor/Office integration types; pure web-interface tools (chatbot websites) are out of scope, because pasting into them is intended use by definition and carries no gap between behaviour and expectation. Software versions and accounts for both phases are listed in Table I.

TABLE I
Experimental environment. Host application versions were recorded after the experiments; because these clients auto-update, earlier host runs may have used slightly earlier builds.

Component	Version / configuration
Browser phase (Kali Linux VM)	
Firefox	140.12.0esr
Grammarly extension	8.937.0 (free account, logged in)
LanguageTool extension	11.0.2 (no account)
mitmproxy	12.2.3
Host phase (Windows 11 25H2)	
Microsoft 365 (Word)	16.0.20326.20144 (Click-to-Run, personal)
Microsoft Edge	153.0.4234.32
DeepL desktop	26.9.1
Grammarly for Windows	1.2.296.1955
iCloud Drive	Advanced Data Protection off

B. Test document and planted secrets

The input is a synthetic "confidential memo" (2,065 characters, 245 words) written to look like an internal compliance document and explicitly marked "DO NOT FORWARD." Embedded in its prose are twelve unique, fictional identifiers: names, an email address, a phone number, project and reference codes, and a UUID canary, `CANARY-BC267061-67DC-485B-8E51-6F5494765CAB`. These identifiers are synthetic and were not intentionally reused from any real source, so any occurrence in captured network traffic is attributable to our input. The canary in particular is a high-specificity attribution marker: its appearance in client-to-server traffic is strong evidence that content from the synthetic document was transmitted toward that service. Table II lists the twelve identifiers and their categories.

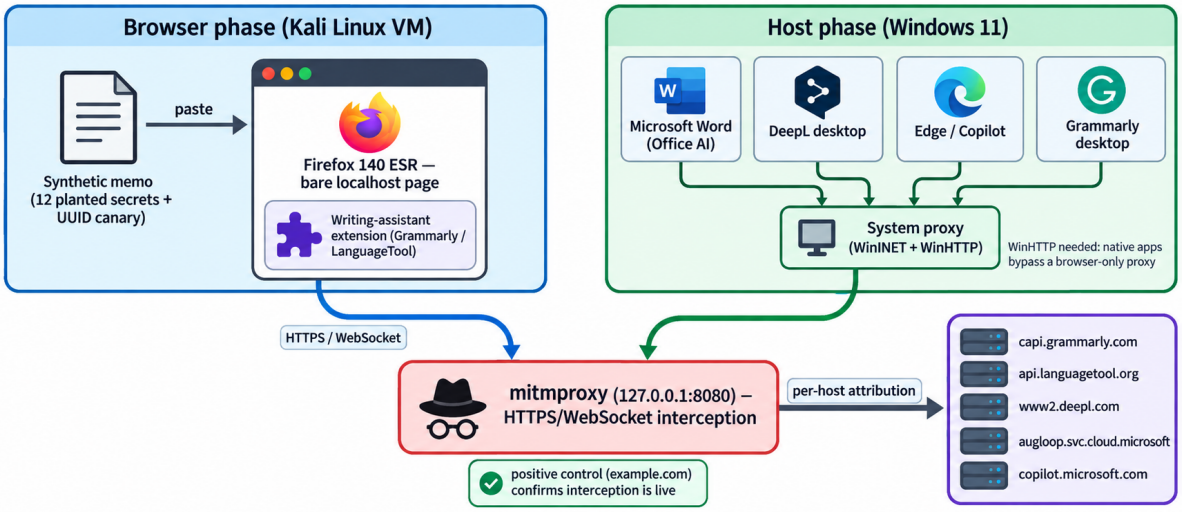


Fig. 1. Experimental setup. In the browser phase (isolated Kali Linux VM), the synthetic memo is pasted into a bare local page and read by the writing-assistant extension. In the host phase (Windows 11), native applications and Office reach the network through the system proxy (WinINET + WinHTTP, the latter required because native clients bypass a browser-only proxy). Both phases route through mitmproxy (`127.0.0.1:8080`); a positive control (example.com) confirms interception is live, and every flow is attributed to its destination host.

TABLE II
THE TWELVE PLANTED IDENTIFIERS EMBEDDED IN THE SYNTHETIC MEMO.

Category	Item
Name (direct identifier)	Helena Voss
Name (direct identifier)	Theodora Baumgartner-Klein
Contact detail	Email address
Contact detail	Phone number
Official / ID number	Employee ID
Official / ID number	Tax registration number
Financial identifier	Q1 expenditure reference
Financial identifier	Reserve fund code
Financial identifier	Budget approval code
Legal / contractual reference	Contract number
Confidential project ID	Project codename
Canary proof-token	Unique UUID leak marker

C. Interception setup

For the browser phase, all captures were performed on an isolated Kali Linux virtual machine so that the man-in-the-middle configuration never touched real browsing. We used mitmproxy [30] (version 12.2.3) as an intercepting HTTPS/WebSocket proxy on `127.0.0.1:8080`. Each tool ran in its own clean Firefox profile with the proxy configured and the mitmproxy certificate authority trusted only within that profile, so decryption was scoped to the experiment. Before any data collection, interception was verified per profile (a known HTTPS site must load without a certificate warning and appear in the proxy); this guards against the most dangerous failure mode in such studies, in which a broken interception silently looks identical to a genuine zero-exposure result.

The document was pasted into a bare local HTML page containing a single text field and nothing else (no third-party scripts, analytics, or other domains) served over `http://localhost` so that extension content scripts activate exactly as they would on a real site, but with zero background noise to filter out.

D. Capture procedure

For each run, the document was loaded into the clipboard and pasted manually into the focused field, after which the run recorded for sixty seconds. This detail matters: a scripted or synthetic paste fills the field visually but does not fire the DOM `input` event the extensions listen for, so the tool never ingests the text and nothing is transmitted, a misleading "clean" zero. A real, manual paste does fire that event, so in our runs the tools genuinely received the document. Each tool was captured for five runs and the baseline for three, following an identical, locked protocol so that between-run variance measures reproducibility rather than procedural drift.

E. Host and system-level phase

Several desktop tools (Microsoft Office, the DeepL desktop client, Edge/Copilot) are not available on Linux, so the host phase was run on a Windows 11 laptop using the same interception principle. mitmproxy ran the same scoring add-on, and both the WinINET and WinHTTP system proxies were pointed at it, with the mitmproxy CA trusted in the Windows certificate store and the proxy enabled only during capture. WinINET carries browser and interactive-app traffic; WinHTTP carries native and background apps. Office and the Grammarly desktop client use WinHTTP and would bypass a WinINET-only (browser) proxy, so covering both is what

makes the host capture complete. For the storage cases, where the question is whether a file is actually read rather than what leaves the network, we additionally recorded local file-access activity with Process Monitor (a Sysinternals tool that logs file, registry, and process events), so we could tell a file being opened from one that was only stored. Each condition used the same synthetic memo and canary. Because a real machine runs several assistants at once, every captured flow was attributed to its destination host, so a transmission is credited to the specific tool responsible rather than to whatever else happened to be running.

F. Metrics

A mitmproxy add-on scored client-to-server HTTP request bodies and client-to-server WebSocket frames; server responses were retained in the raw capture for validation but excluded from exposure scoring, so a backend echoing text back cannot inflate the figure. The raw captures were also archived so they can be re-analysed offline. We report, in order of importance:

Planted-secret count (primary).
How many of the twelve identifiers reached the servers, verified individually. The canary makes this unambiguous.

Exposure %.
Character coverage: the fraction of the document's character positions covered by at least one matching 20-character window, so that overlapping matches contribute each position at most once. Let $\mathcal{T}$ be the set of decoded outbound substrings and $w_{20}(i)$ the 20-character window at position i; the covered set and exposure are

$$C = \bigcup_{i:\, w_{20}(i) \in \mathcal{T}} \{i, \ldots, i+19\},$$

$$\text{Exposure}\,(\%) = \frac{|C|}{N} \times 100,$$

where N is the document length in characters. A 20-character window is long enough to avoid false matches from common words and short enough to catch fragmented sends; a per-host concatenated-transcript pass (all of a host's traffic joined into one stream) additionally catches content split across multiple frames. Matching is case-folded and performed over plain, URL-decoded, JSON-unescaped, HTML-entity, base64-decoded, UTF-16, and Unicode-NFC variants so that encoded transmission is still detected, and compressed bodies (gzip, brotli, zstd) are decoded first. Traffic to hosts also seen in the no-extension baseline is discarded only when it carries no document coverage and no secret (pure background); a matching flow on a host shared with the baseline is never dropped, so baseline subtraction removes noise without erasing a genuine transmission.

Reproducibility and confidence.
We report confidence in two parts: *repeatability*, the standard deviation of exposure % across a condition's runs in percentage points (pp), labelled High (< 3 pp), Medium (3–10 pp), Low (> 10 pp), or not assessable for single-run conditions; and *capture completeness*, complete when no TLS handshake escaped interception and a lower bound otherwise.

Traffic visibility.
Reported as two separate numbers: the share of intercepted events that used HTTPS, and the count of TLS handshakes (the initial exchange that sets up an encrypted connection) that could not be intercepted, for example due to certificate pinning, where an application accepts only its own expected certificate so an intercepting proxy cannot decrypt that connection. These are never combined into a single figure, because a non-zero failure count means the exposure result is a lower bound (traffic we cannot decrypt could only add to what was sent, never subtract).

Beyond the twelve planted secrets, we also run an automatic PII detector (Microsoft Presidio, an open-source tool that flags names, e-mail addresses, ID numbers, and similar) over the decrypted bodies, to catch sensitive data we did not plant.

Where within-setup runs were identical we report the measured exposure directly rather than as a confidence interval; such run-to-run stability is not a claim of stability across software versions, accounts, regions, document types, or future server-side behaviour.

V. Results

A. Whole-document transmission in the browser phase

Table III summarises every measured condition and its receiving host across both phases; this subsection covers the browser phase, and the host phase follows in Section V-E. Both extensions transmitted essentially the entire document and **all twelve planted secrets, including the canary**, to their servers. Grammarly transmitted a mean **99.0%** of the document's characters; LanguageTool transmitted **91.9%**. Every run of each tool produced the same measured exposure (standard deviation 0.0 pp), indicating high within-setup repeatability rather than determinism across environments. The no-extension baseline transmitted **0.0%** and none of the secrets. All transmission used HTTPS, and there were zero un-interceptable TLS handshakes, so these browser-phase figures are complete, not lower bounds; host-phase figures can be lower bounds where a client pins its certificate (Section VII). The transmission is not confined to one class of data: both tools transmitted every information type in the document (personal names, contact details, official identification numbers, financial codes, a legal contract reference, the confidential project codename, and the canary), and formatted identifiers such as ID numbers were sent as readily as ordinary sentences. The per-tool repeatability verdicts appear in Table III alongside the exposure figures; capture was complete for every browser-phase condition and is a lower bound only where a client pinned its certificate (Edge/Copilot, marked †).

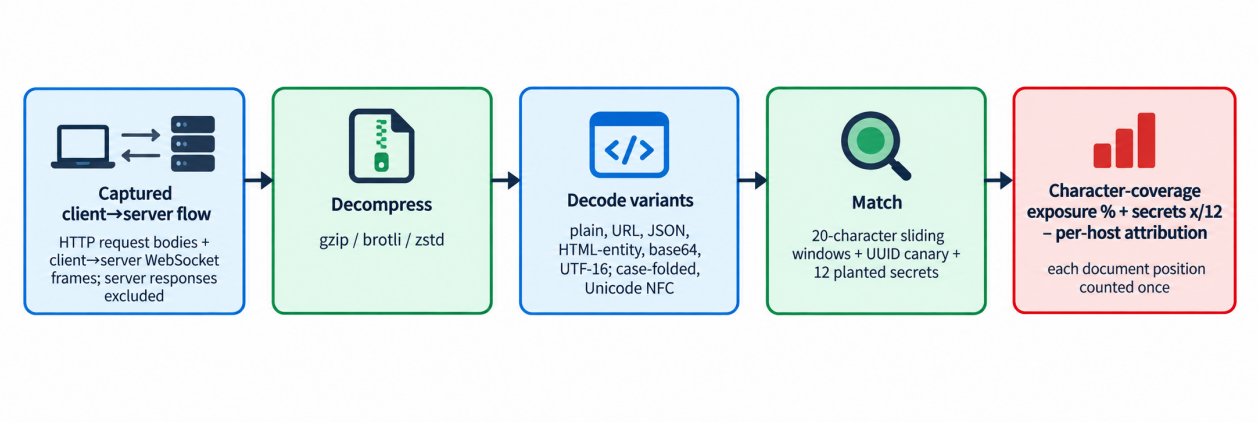


Fig. 2. Exposure-scoring pipeline. Each outbound flow is decompressed, decoded across encodings, and matched against the document with a 20-character sliding window plus the canary and twelve secrets, giving a coverage percentage, a secret count, and the receiving host.

TABLE III
DEFAULT-CONFIGURATION EXPOSURE AND CONFIDENCE PER CONDITION. EXPOSURE IS CHARACTER COVERAGE, AVERAGED ACROSS RUNS; HOST-PHASE ROWS ARE EXPLORATORY AND READ INDIVIDUALLY. †EDGE/COPILOT IS A LOWER BOUND (CERTIFICATE PINNING).

Condition	Exposure	Secrets	Repeatability	Receiving host
Browser phase (paste into an ordinary web field)				
Grammarly	**99.0%**	**12/12**	High (5 runs)	`capi.grammarly.com` (WebSocket)
LanguageTool	**91.9%**	**12/12**	High (5 runs)	`api.languagetool.org`
No extension (baseline)	0.0%	0/12	High (3 runs)	—
Host phase (desktop apps and opening a received file; exploratory)				
DeepL desktop (paste)	**99.6%**	**12/12**	High (3 runs)	`www2.deepl.com`[a]
Word: connected experiences on, open `.docx`	**96.8%**	**12/12**	High (3 runs)	`augloop.svc.cloud.microsoft`
Word Protected View + Grammarly desktop	**94.2%**	**12/12**	High (2 runs)	`capi.grammarly.com`
Edge Copilot: summarise PDF	**76.9%**†	**12/12**	High (3 runs)	`copilot.microsoft.com`
Word: connected experiences off, open `.docx`	0.0%	0/12	N/A (N=1)	(socket opens, no content)
Passive: USB / iCloud sync / non-AI viewer	0.0%	0/12	N/A (N=1 each)	(no document content on tool hosts)

[a] `www2` is simply a secondary/service hostname, distinct from the public `www` site; here it is the host the DeepL client uploads text to.

B. What was transmitted, and how

Grammarly streamed the document to its backend over a WebSocket connection (about 5–6 kB of client-to-server payload per run) reaching a single content-bearing host; LanguageTool sent it as an HTTPS `POST` to its own check endpoint (5.6 kB to one host). These outbound-volume and external-host figures show the document leaves in a small number of requests to the tool's own server, not scattered across many hosts. In both cases the twelve planted identifiers, including the canary, were recovered from the decrypted client-to-server traffic. The ∼8 percentage-point gap between the two tools is a coverage artefact, not a difference in whether sensitive content leaves: the characters LanguageTool did not cover are almost all newlines and short header or salutation lines (for example the `Date:` line and the "Dear Marcus," greeting) that it sends as separate segments too short to fill a 20-character window. Excluding whitespace, coverage is 98.9% (Grammarly), 93.6% (LanguageTool), and 82.2% (Edge/Copilot). We did not isolate Edge's remaining residual; it is consistent with differences in how the PDF text is extracted (reflow, hyphenation, and ligatures) rather than omission of content, and its pinned channels make that figure a lower bound in any case. All three transmitted every one of the twelve secrets.

The transmission is a single outbound burst at the moment of paste, after which the connection falls quiet (all tool-host frames arrived within about five seconds of paste, then nothing for the rest of the 60 s capture); the exposure is input-triggered, not continuous. The no-extension baseline is instructive: it generates more total traffic than either tool run (megabytes of browser and OS background activity), yet none of it carries document content, which is why a baseline is needed, to separate the environment's own chatter from document-bearing traffic.

C. Beyond the planted secrets: account linkability

Grammarly's outbound WebSocket carried, alongside the document, a service authentication token and per-session account identifiers, so the transmitted text is linkable to a specific Grammarly account; LanguageTool's requests carried no such account identifier. We did not test cross-session or cross-service linkage, and the token travels over the client's own encrypted connection to its backend, so this is a note on linkability rather than a credential-disclosure finding. We

surfaced it with the automatic PII pass (Presidio) noted in Section IV-F.

D. Independent corroboration without a proxy

Our main method routes the browser's traffic through the mitmproxy so we can decrypt and read it. A fair worry is that inserting the proxy could itself change the tool's behaviour, making the upload an artefact of our setup rather than something that happens normally. To check, we repeated the paste with the proxy removed completely: Grammarly used a normal, direct internet connection while `tcpdump` (a tool that records which servers the computer contacts, and when) logged the traffic. At the moment of paste, the browser still contacted Grammarly's own servers (an AWS backend and Grammarly content-delivery edges) and uploaded several kilobytes. Without the proxy the traffic is encrypted, so we cannot read its contents, but the destination and timing show the upload happens even with nothing intercepting it. The proxy only observes the transmission; it does not cause it.

E. Desktop and system-level exposure: an action spectrum

We additionally conducted a secondary, exploratory host phase to examine whether document exposure also occurs across other integration surfaces, using the host interception setup of Section IV. Unlike the primary browser experiment, these host conditions involve different user actions and document formats, so they should not be read as direct tool-to-tool comparisons; each is an observation about one surface. The results still arrange along a spectrum of user action (Figure 3); the prose below highlights only the findings that change the interpretation, and the receiving host for each is listed in Table III.

Active submission transmits the document, as in the browser phase. The **DeepL** desktop client, given the document by paste, transmitted essentially all of it (99.6% mean over three runs) and every secret to DeepL's service backend; the entire document was uploaded, not only the portion returned as a translation.

Invoking Copilot on an opened PDF transmits document content. Invoking **Edge's Copilot** to summarise the planted PDF transmitted **76.9%** of the file and all twelve secrets to Microsoft's Copilot backend (three runs; runs 1–2 asked for a summary and run 3 for "interesting facts," and the transmitted content was identical). Certificate-pinned handshakes escaped interception here, so 76.9% is a lower bound. The coverage is lower than the paste-based tools because the source was a PDF: Edge transmits its text extraction, which drops layout and formatting characters, yet every one of the twelve secrets, canary included, was sent. Run 3 is instructive: Copilot's visible reply stated it would answer without exposing the confidential details, yet the capture shows document content, including all twelve secrets and the canary, had already been sent to Microsoft. A model declining to display a secret is therefore not evidence that the secret was not transmitted; the privacy boundary is the network, not the chat reply.

Opening a document in Word transmits it. In the tested Microsoft 365 configuration (personal account, connected experiences enabled), opening the same document as a `.docx`, with no paste and no submission, streamed **96.8%** of it and every secret to Microsoft's "Augmentation Loop" (augloop), a Microsoft connected-experience endpoint, via posts to an `OfficeCopilotOrchestrationWorkflow`; augloop returned the same 96.8% in every connected-experiences-enabled run. Microsoft documents that connected experiences analyse Office content in the cloud, are available unless the relevant control is disabled, and can be turned off through an Office privacy setting [31, 32]. The transmission therefore came from the default productivity suite itself, not an add-in. We measured a consumer licence; the EU Data Boundary and processor-agreement commitments Microsoft offers apply to enterprise tenants and do not describe this configuration. Other Microsoft hosts seen during these runs (`activity.windows.com`, `arc.msn.com`, `ecs.office.com`, template and consent CDNs) carried telemetry or configuration traffic, not document content; augloop was the only content-bearing channel.

Disabling one setting removes the channel (single-run mitigation). Turning off the single Office setting "experiences that analyse your content" (File $>$ Account $>$ Account Privacy $>$ Manage Settings, under Connected Experiences) and reopening the same file reduced augloop's transmission to **0%** (0/12), with the socket still opening but carrying no document content. This single disabled-control run supports attributing the transmission to the Office connected-experiences feature rather than to the act of opening a file, and points to a concrete one-switch mitigation. It removes Microsoft's own channel only, however: a third-party assistant hooking the same window is unaffected, as the Protected View result next shows.

Protected View does not stop a third-party assistant. When the file carried a mark-of-the-web and opened in Word's sandboxed Protected View (the yellow "Enable Editing" banner still showing, no user action), Microsoft's own augloop stayed silent, Protected View contained it, but the **Grammarly** desktop client still transmitted **94.2%** of the document and every secret to its backend (two runs, identical). With augloop silent, `capi.grammarly.com` was the only content-bearing host in these runs, so per-host attribution credits the transmission to the Grammarly desktop client. We observed the transmission but did not determine the mechanism by which the client obtained the text. Protected View thus sandboxes macros and even Microsoft's own AI, but not an independent assistant reading the same window.

A file only stored on a USB or cloud drive: no document content observed. A distinct question is whether a confidential file is transmitted simply by being stored on a removable drive or a synced cloud folder while the writing assistants run in the background. It is not. Inserting the USB without opening the file (Process Monitor showed only the Windows indexer and shell probing volume metadata, not the file being read) and dropping the file into iCloud Drive to sync each

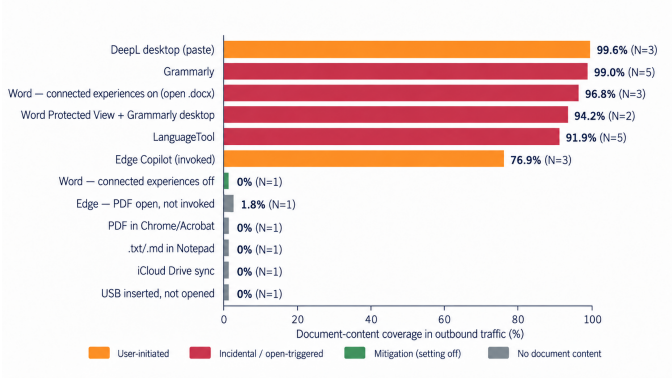


Fig. 3. Document-content coverage in outbound traffic, by user action and surface. Exact run counts N are shown per bar; single-run rows are individual observations, not repeatability estimates. Edge/Copilot is a lower bound (some channels were certificate-pinned). The "no document content" class means no attributable document-specific content: the passive Edge bar's 1.8% is a generic boilerplate phrase, not a planted secret.

transmitted **no** document content to the writing tools' servers (0/12), even though Grammarly and the other clients were running throughout: the file never entered a surface they watch. iCloud's own sync channels were reachable in the capture but carried no readable document text (we saw the connections but no cleartext content): the file syncs to Apple encrypted in transit (Advanced Data Protection, which would add end-to-end encryption, was off in our setup) [33], so it reaches Apple's cloud but is never exposed to the LLM writing tools. The same held for opening the file in a viewer with no cloud-AI integration and no assistant attached (Notepad, Adobe Acrobat, or Chrome's PDF viewer), and for opening the PDF in Edge without invoking Copilot (0/12 secrets; the passive Edge case matched only 1.8% of the document's characters, a single 37-character run of the generic phrase "personally identifiable information" with no planted secret, i.e. incidental boilerplate rather than document content). Consumer antivirus (**Avast**) is a separate class rather than an LLM tool: it uploaded an unknown executable to its cloud (CyberCapture) plus reputation and telemetry, but not the document's contents.

Exposure is surface-dependent, not storage-dependent. A file on a USB stick or in a synced cloud folder is not read by the writing tools until its text enters a surface they watch, so passively receiving or opening a file transmits nothing unless that surface has a cloud-AI integration watching the content. Across the conditions we tested, content leaves the machine when it enters such a surface, whether the user pastes into a writing assistant, invokes an assistant, or opens the file in a cloud-AI-enabled editor. Table IV answers the research question directly: for each surface it gives the trigger, the user action required, and whether an off-switch exists.

F. Permissions and the exposure channel

A natural question is whether these tools reach beyond text into device sensors. Auditing the Windows capability store (mic, camera, location, contacts, full-disk access, and the rest) and each extension's declared permissions showed that none of the writing or translation tools (Grammarly, LanguageTool, DeepL) holds microphone, camera, or broad-filesystem access: the microphone and camera belong only to communication, voice, and meeting apps. What the writing tools do hold is the browser's "Access your data for all websites" host permission, which is precisely the capability that lets a content script read every editable field [34]. The exposure channel is therefore the text a user types or pastes, granted through this all-sites content permission, not audio or video surveillance. In our setup the always-on clients (Grammarly for Windows, DeepL desktop) launched at start-up, so they are already running before the user does anything.

VI. Discussion

The clearest single result is the canary, not the percentage. A unique token that existed only inside a document marked "DO NOT FORWARD" was found, seconds after an ordinary paste, in traffic bound for two companies' servers, placed there by software the user installed once and forgot. Server-side analysis requires the text to be uploaded, and both vendors document that they process user text [35, 36]. The finding is the gap between that design and what a user, told "don't share this with AI," would expect when they paste a paragraph into an unrelated field, a gap best understood as a breach of contextual integrity, in which information flows to a context the user did not intend [37].

This distance matters because the exposure is (a) invisible: there is no prompt, no click, no confirmation; (b) complete: essentially the whole document, not a fragment; and (c) highly repeatable within the tested setup: it happened on every run. For an individual handling regulated data (health, legal, financial) the implication is direct: the writing assistants we tested are incompatible with a "do not share" instruction unless explicitly disabled, and disabling them is not the default. It also matters when a tool sends: the active, on-demand transfers and the incidental, background ones distinguished in Section II carry different risk, and the grammar checkers we tested are the silent, background kind that transmit the moment text appears. Table IV summarises this across surfaces: exposure differs little in extent once content enters a watched surface, but the trigger, the required user action, and the available off-switch differ markedly.

A. Regulatory context (EU GDPR)

Our tests use only synthetic data, so no real personal data is involved; but the same mechanism matters wherever those fields would hold real personal data. A silent whole-document upload raises questions under three expectations of the EU's data-protection law (GDPR, Regulation (EU) 2016/679): being transparent about where data goes, sending only the data that is needed, and informing the person the data is about of the third parties that receive it (Arts. 5, 13, and 14). If the receiving servers are outside the EU, the law's cross-border-transfer rules (Chapter V) also apply. The risk is largest at work, where an employee may paste client data into a consumer tool that has no data-processing agreement in place (Art. 28). We make no

TABLE IV
ANSWER TO THE RESEARCH QUESTION: HOW EXPOSURE DIFFERS ACROSS SURFACES — THE TRIGGER, THE USER ACTION, AND WHETHER AN OFF-SWITCH EXISTS (RECEIVING HOSTS ARE IN TABLE III).

Surface	Trigger	User action	Off-switch
Grammarly (browser extension)	incidental	none beyond typing/paste	disable/remove extension
LanguageTool (browser extension)	incidental	none beyond typing/paste	disable/remove extension
DeepL desktop	user-initiated	paste into the app	n/a (user-initiated)
Word (connected experiences)	incidental	open the file	one Office setting
Edge + Copilot	user-initiated	invoke Copilot	n/a (user-initiated)
Grammarly desktop (in Word)	incidental	open the file	quit/remove client
Passive (USB / iCloud / plain viewer)	none	—	n/a (no transmission)

legal ruling — whether any specific use is lawful depends on the details — but the engineering point is simple: the problem is not only that the data leaves, but that it leaves invisibly, with nothing telling the user it happened.

B. Broader impact

We raise these implications as a matter of professional responsibility: engineering codes of ethics ask those who build and study such systems to hold paramount the public's safety, welfare, and privacy [38]. Our experiment measures transmission, not what happens afterwards. Once document content leaves the machine it enters the downstream ecosystem of retention, provider-side processing, model training, and attribute inference reviewed in Section III, expanding the set of systems that can process, retain, and act on it. These downstream practices vary by provider and configuration and lie outside our measurement scope; quantifying the network exposure precisely, rather than trusting vendor assurances, is the necessary first step to reasoning about them.

VII. LIMITATIONS

We measured across several surfaces (browser extensions on an isolated Linux VM; desktop, Office, and operating-system tools on Windows 11, since several desktop tools are unavailable on Linux), which broadens the range of surfaces examined. Two limits apply. First, traffic protected by certificate pinning cannot be read (for example `*.events.data.microsoft.com` and some Adobe and Google endpoints); content we cannot decrypt is not counted, so figures from conditions with pinned channels (Edge/Copilot) are lower bounds, whereas the browser-phase conditions had zero un-interceptable handshakes and are complete. Interception can itself weaken the observed connection [39]; the proxy CA was trusted only in the browser test profile and, for the host phase, added to the Windows certificate store for the capture window only, with the raw traffic reported unmodified.

Second, a native application can use sockets that ignore the system proxy: the Grammarly Windows client on Notepad sent only telemetry through the proxy while opening a direct connection we could not inspect, so that case is reported as unmeasured rather than zero; results captured through the proxy (browser extensions, Office, and Grammarly in Word) are unaffected.

VIII. ETHICS

All planted test data is synthetic: no real person, organisation, or system is referenced. The study measures default-configuration behaviour of legitimately installed software on the researcher's own machine and does not attack the software vendors' systems or other users. Because the host phase intercepted system-wide traffic, incidental background flows from the researcher's own accounts were also decrypted; only flows relevant to the synthetic-document experiment were retained for analysis and unrelated incidental background flows were excluded, the proxy CA was added to the Windows certificate store only for the capture window and removed afterwards, and the published capture summaries were scrubbed of incidental account identifiers before release. No responsible-disclosure step was required: the behaviour we measure is documented by the software vendors and is not a vulnerability.

IX. CONCLUSION AND FUTURE WORK

This project measured how much of a confidential document leaves the machine when common LLM-integrated productivity tools are used at their defaults. In the primary browser experiment, both writing-assistant extensions transmitted all twelve planted identifiers and nearly all measurable document content, while the identical no-extension baseline transmitted none. The secondary host probes show that comparable exposure can occur through other productivity surfaces, but the triggering interaction varies: some transfers follow explicit user invocation (the DeepL paste, the Copilot summary), others occur when content enters a background or editor-integrated service (the browser assistants, Word's connected experiences), and the negative controls show that storage location alone does not cause exposure. As an answer to the research question, exposure differs little in extent once content reaches a watched surface—near-total, with all twelve secrets—and differs mainly in what triggers it and whether the user can switch it off. The tools behave as designed; the finding is how far that design sits from the expectation of a user told not to share the file. The unifying result is that exposure is surface-dependent: content leaves the machine when it enters a surface an LLM-integrated service watches, not because of where the file is stored. This moves the user's real point of control to before the text enters such a surface. That turning off one Office setting removed the largest passive

transfer, a single-run observation, indicates the behaviour is a configuration choice, not a technical necessity.

Future work. A natural next step is to classify the transmitted data by type: an automatic labelling of which categories of personal information (names, contact details, financial data, identifiers) each tool transmits. This would extend the automated PII sweep with Microsoft Presidio [40], already used here to surface account identifiers travelling alongside the document, into a full per-category inventory across the captures. A further mechanistic extension remains: dynamic instrumentation (for example Frida) to read certificate-pinned channels that are currently only a lower bound.

Data Availability

The capture and analysis code, the synthetic memo, and the planted-secret list are available at https://github.com/MarawanEldeib/privacy-analysis-research.

References

[1] Oleksii Starov and Nick Nikiforakis. Extended tracking powers: Measuring the privacy diffusion enabled by browser extensions. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 1481–1490, 2017.

[2] Michael Weissbacher, Enrico Mariconti, Guillermo Suarez-Tangil, Gianluca Stringhini, William Robertson, and Engin Kirda. Ex-ray: Detection of history-leaking browser extensions. In *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*, pages 590–602, 2017.

[3] Anupama Aggarwal, Bimal Viswanath, Liang Zhang, Saravana Kumar, Ayush Shah, and Ponnurangam Kumaraguru. I spy with my little eye: Analysis and detection of spying browser extensions. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 47–61, 2018.

[4] Quan Chen and Alexandros Kapravelos. Mystique: Uncovering information leakage from browser extensions. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1687–1700, 2018.

[5] Aurore Fass, Dolière Francis Somé, Michael Backes, and Ben Stock. DoubleX: Statically detecting vulnerable data flows in browser extensions at scale. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021.

[6] Nicholas Carlini, Adrienne Porter Felt, and David Wagner. An evaluation of the Google Chrome extension security architecture. In *Proceedings of the 21st USENIX Security Symposium*, 2012.

[7] Nikolaos Pantelaios, Nick Nikiforakis, and Alexandros Kapravelos. You've changed: Detecting malicious browser extensions through their update deltas. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020. doi: 10.1145/3372297.3423343.

[8] Asmit Nayak, Rishabh Khandelwal, Earlence Fernandes, and Kassem Fawaz. Experimental security analysis of sensitive data access by browser extensions. In *Proceedings of the ACM Web Conference (WWW)*, 2024.

[9] Qinge Xie, Manoj Vignesh Kasi Murali, Paul Pearce, and Frank Li. Arcanum: Detecting and evaluating the privacy risks of browser extensions on web pages and web content. In *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[10] Duc Bui, Brian Tang, and Kang G. Shin. Detection of inconsistencies in privacy practices of browser extensions. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, pages 2780–2798, 2023. doi: 10.1109/SP46215.2023.10179338.

[11] Savino Dambra et al. When sally met trackers: Web tracking from the users' perspective. In *Proceedings of the 31st USENIX Security Symposium*, 2022.

[12] Steven Englehardt and Arvind Narayanan. Online tracking: A 1-million-site measurement and analysis. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016. doi: 10.1145/2976749.2978313.

[13] Asuman Senol, Gunes Acar, Mathias Humbert, and Frederik Zuiderveen Borgesius. Leaky forms: A study of email and password exfiltration before form submission. In *Proceedings of the 31st USENIX Security Symposium*, pages 1813–1830, 2022.

[14] Munir, Demir, Li, Kollnig, and Shafiq. Every keystroke you make: A tech-law measurement and analysis of event listeners for wiretapping, 2025. arXiv preprint arXiv:2508.19825.

[15] Oleksii Starov, Phillipa Gill, and Nick Nikiforakis. Are you sure you want to contact us? Quantifying the leakage of PII via website contact forms. In *Proceedings on Privacy Enhancing Technologies (PoPETs)*, pages 20–33, 2016. doi: 10.1515/popets-2015-0028.

[16] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010.

[17] Jingjing Ren, Ashwin Rao, Martina Lindorfer, Arnaud Legout, and David Choffnes. ReCon: Revealing and controlling PII leaks in mobile network traffic. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2016. doi: 10.1145/2906388.2906392.

[18] Andrea Continella, Yanick Fratantonio, Martina Lindorfer, Alessandro Puccetti, Ali Zand, Christopher Kruegel, and Giovanni Vigna. Obfuscation-resilient privacy leak detection for mobile apps through differential analysis. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017. doi: 10.14722/ndss.2017.23465.

[19] Yash Vekaria, Aurelio Canino, Jonathan Levitsky, Alex

Ciechonski, Patricia Callejo, Anna Maria Mandalari, and Zubair Shafiq. Big help or big brother? Auditing tracking, profiling and personalization in generative AI assistants. In *Proceedings of the 34th USENIX Security Symposium*, 2025.

[20] Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *Proceedings of the 30th USENIX Security Symposium*, 2021.

[21] Milad Nasr, Javier Rando, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Florian Tramèr, and Katherine Lee. Scalable extraction of training data from aligned, production language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2311.17035.

[22] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Analyzing leakage of personally identifiable information in language models. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, pages 346–363, 2023.

[23] Robin Staab, Mark Vero, Mislav Balunović, and Martin Vechev. Beyond memorization: Violating privacy via inference with large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.07298.

[24] Niloofar Mireshghallah, Hyunwoo Kim, Xuhui Zhou, Yulia Tsvetkov, Maarten Sap, Reza Shokri, and Yejin Choi. Can LLMs keep a secret? Testing privacy implications of language models via contextual integrity theory. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.17884.

[25] Shuai Cheng, Haitao Xu, Shu Meng, Shuai Hao, Chuan Yue, and Zhao Li. The privacy paradox of LLMs: User perceptions and the reality of PII leakage. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, CHI '26. ACM, 2026. doi: 10.1145/3772318.3791809.

[26] Anthropic. Detecting and countering misuse of AI: September 2026. Technical report, Anthropic, September 2026. Accessed 2026-09-18. Threat intelligence report.

[27] Shresta Bellary Seetharam, Nabeel Mohamed, Billy Melicher, Oleksii Starov, Qinge Xie, and Fang Liu. That AI extension helping you write emails? It's reading them first. Technical report, Palo Alto Networks Unit 42, 2026. https://unit42.paloaltonetworks.com/high-risk-gen-ai-browser-extensions/ Accessed 2026-09-18.

[28] Cloud Security Alliance. AI browser extensions: The DLP-invisible enterprise attack surface. Technical report, Cloud Security Alliance, April 2026. CSA research note, 11 April 2026. https://labs.cloudsecurityalliance.org/research/csa-research-note-ai-browser-extension-security-gaps-2026041/ Accessed 2026-09-18.

[29] Cyberhaven Labs. 2026 AI adoption & risk report. Technical report, Cyberhaven, 2026. https://www.cyberhaven.com/resources/report/ai-adoption-risk-report-2026 Accessed 2026-09-18.

[30] Aldo Cortesi, Maximilian Hils, Thomas Kriechbaumer, and mitmproxy contributors. Mitmproxy: A free and open source interactive HTTPS proxy, 2010–2026.

[31] Microsoft. Connected experiences in Office. Microsoft Learn, 2026. States that connected experiences analyse Office content in the cloud and are available unless the relevant control is disabled. https://learn.microsoft.com/en-us/microsoft-365-apps/privacy/connected-experiences Accessed 2026-09-18.

[32] Microsoft. Data, privacy, and security for Microsoft Copilot. Microsoft Learn, 2026. Updated 2026-08-18. https://learn.microsoft.com/en-us/microsoft-365/copilot/microsoft-365-copilot-privacy Accessed 2026-09-18.

[33] Apple Inc. Apple platform security. Apple Support, 2024. iCloud data security and Advanced Data Protection. https://support.apple.com/guide/security/welcome/web.

[34] Mozilla. Manage extension permissions (Firefox). Mozilla Support, 2024. "Access your data for all websites" lets an extension read and change page content. https://support.mozilla.org/kb/permission-request-messages-firefox-extensions Accessed 2026-09-18.

[35] Grammarly. How we protect your privacy. grammarly.com, 2025. https://www.grammarly.com/privacy Accessed 2026-09-18.

[36] LanguageTool. Privacy policy. languagetool.org, 2025. https://languagetool.org/legal/privacy Accessed 2026-09-18.

[37] Helen Nissenbaum. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–157, 2004.

[38] IEEE. IEEE code of ethics. IEEE Policies, Sec. 7.8, 2020. https://www.ieee.org/about/corporate/governance/p7-8.html.

[39] Zakir Durumeric, Zane Ma, Drew Springall, Richard Barnes, Nick Sullivan, Elie Bursztein, Michael Bailey, J. Alex Halderman, and Vern Paxson. The security impact of HTTPS interception. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.

[40] Microsoft. Presidio: Data protection and de-identification SDK. https://github.com/microsoft/presidio, 2018. Open-source software; docs at https://microsoft.github.io/presidio/.

Declaration of Originality / Eigenständigkeitserklärung

I hereby declare that this work is entirely my own and that I have used no sources or aids other than those indicated. All passages taken verbatim or in substance from other sources are identified as such. This work has not previously been submitted, in the same or a similar form, to any examination authority, and has not yet been published. All experimental data reported here are synthetic; no real personal data were used.

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus anderen Quellen übernommen wurden, sind als solche kenntlich gemacht. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt und ist noch nicht veröffentlicht worden.

Stuttgart, September 22, 2026

Marawan Eldeib